



PROJET VLS NFE 204

Circuit de validation

	Nom		Date	Visa
Auteur	Olivier JEANTET / Yiv NGOU		27/01/2015	

Historique

Version	Date	Description	Auteur
V0.1	27/01/2015		Olivier JEANTET / Yiv NGOU
V0.2	28/02/2015	Ajout des fonctionnalités des nœuds Riak, scripts etc.	Yiv NGOU

SOMMAIRE

1. Introduction.....	3
2. Description de solution.....	3
3. Schéma.....	5
4. Installation de Raspbian	6
5. Installation de Riak.....	9
6. Construire les noeuds Riak.....	11
7. Insertion des données dans les nœuds Riak	13
8. Quelques statistiques	13
9. Conclusion	15
10. Scripts shell et python	15

1. Introduction

Le projet choisi est la collecte des données VELIB quotidiennement les trajets effectués en Vélib, sous la forme de documents JSON à partir du site de JC Decaux <https://developer.jcdecaux.com/#/opendata/vls> que nous proposons d'effectuer.

Nous allons réaliser un script en Python qui va récupérer ces données quotidiennement.

Ces données sont prélevées à un intervalle régulier et seront subies à divers traitement pour produire les statistiques de volumes d'emprunts, de retours et de disponibilités des vélos.

A partir de ces données collectées, nous ferons des prédictions pour l'heure à venir.

2. Description de solution

Un système de vélos en libre-service (VLS) met à disposition du public des vélos gratuitement ou non. Ce service de mobilité permet d'effectuer des déplacements de proximité principalement en milieu urbain. Cette location de vélos est une forme de consommation collaborative et permet ainsi de lever trois freins à la pratique du vélo : le stationnement à domicile, le vol et la maintenance de son vélo personnel (source wikipedia).

Nous proposons de créer une base documentaire à partir des données mises en ligne par le site de JC Decaux. Ces données sont importées au format JSON.

Selon la définition de l'api JC Decaux nous avons :

- les données statiques (position GPS de la station, nom et l'adresse de la station)*
- les données dynamiques nécessitant l'utilisation d'une clé d'API (pour des raisons légales) donne les chiffres en temps réel de la station (ouverte ou fermée, nombres de vélos disponibles etc.), soit en format JSON :*

```
{ "number":int,  
  "name": string,  
  "address":string,  
  "position":{"lat":double,"lng":double},  
  "banking":boolean,  
  "bonus":boolean,
```

```
"status":enumerated,  
"contract_name":string,  
"bike_stands":int,  
"available_bike_stands":int,  
"available_bikes":int,  
"last_update":timestamp}
```

A partir de ces données prélevées par tranche horaire chaque jour :

- *nous effectuons des statiques par heure et par jour de la semaine pour le volume d'emprunts, le volume de disponibilités par station.*
- *pour les prédictions, nous utilisons dans le stockage des documents:*

Lorsqu'un utilisateur veut connaître la disponibilité d'une station pour la fin de l'heure en cours :

$$hp = (M\ h) * [1 + ((h-1)-(M\ h-1)/(M\ h-1))]$$

avec :

hp : disponibilité prévue pour la fin de l'heure en cours

h-1 : disponibilité de l'heure précédente

M h : Moyenne des disponibilités concernant la même heure du même jour (de la semaine) pour les 4 dernières semaines

M h-1 : Moyenne des disponibilités concernant l'heure précédente du même jour (de la semaine) pour les 4 dernières semaines.

Il s'agit donc de prédire la disponibilité d'une station à la fin de l'heure, en indiquant sa disponibilité en début d'heure pondérée de l'évolution de sa disponibilité de l'heure précédente par rapport à sa disponibilité moyenne pour ce jour de la semaine à cette même heure (calculée sur les 4 dernières semaines).

Etant donné la mise à jour non synchronisé par contrat, nous proposons d'acquérir les données par station et par contrat par intervalle de temps déterminé sous la forme de la requête http :

https://api.jcdecaux.com/vls/v1/stations/XXXXX?contract=Paris&apiKey=Clef_api

XXXXX : numéro de la station

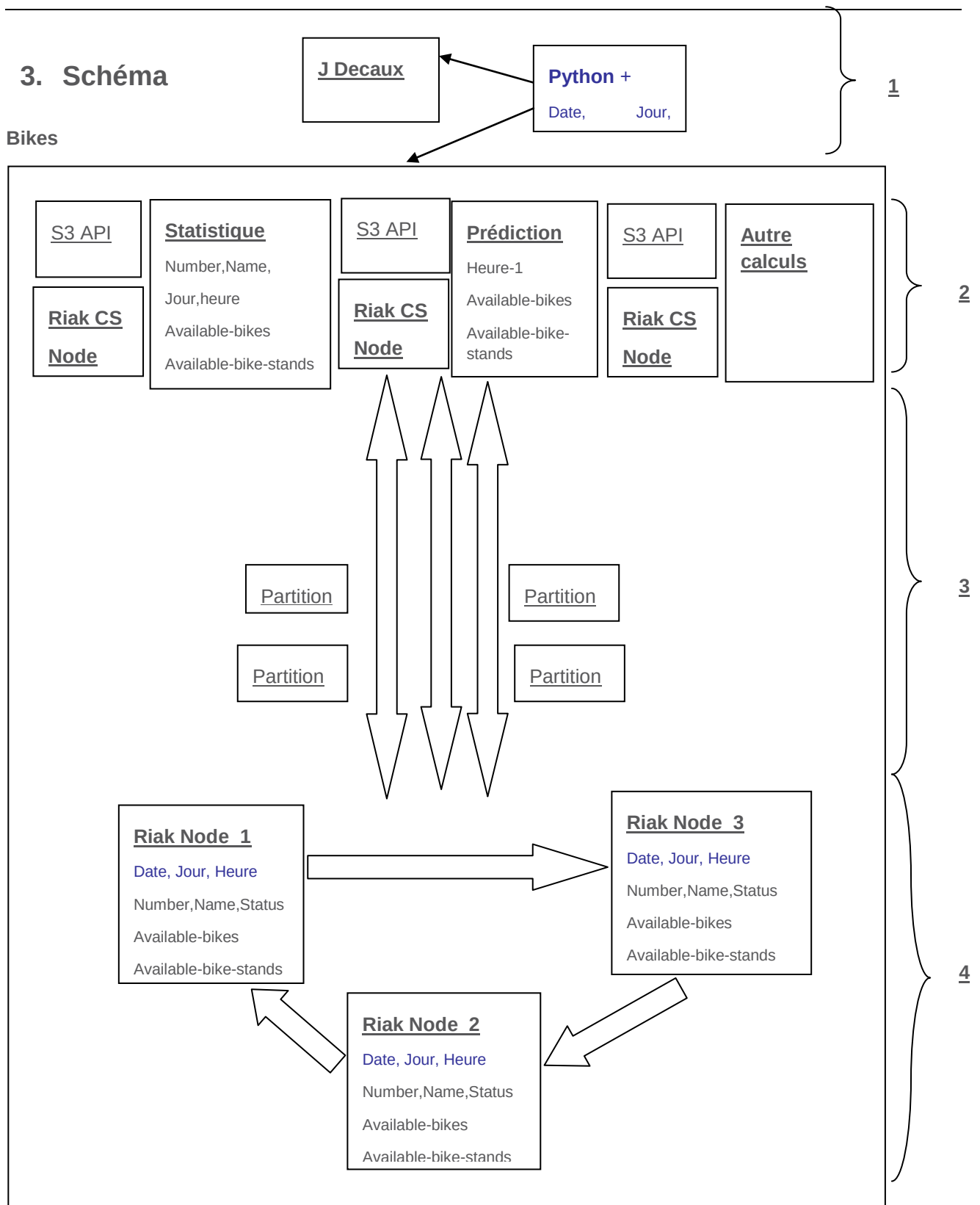
Clef_api : clé de l'api OPENDATA JC Decaux

Cela permet de faire la répartition de charges par nœud, sur une ou plusieurs machines.

L'étude sera faite dans un autre système que MongoDB à savoir RIAK.

3. Schéma

Bikes



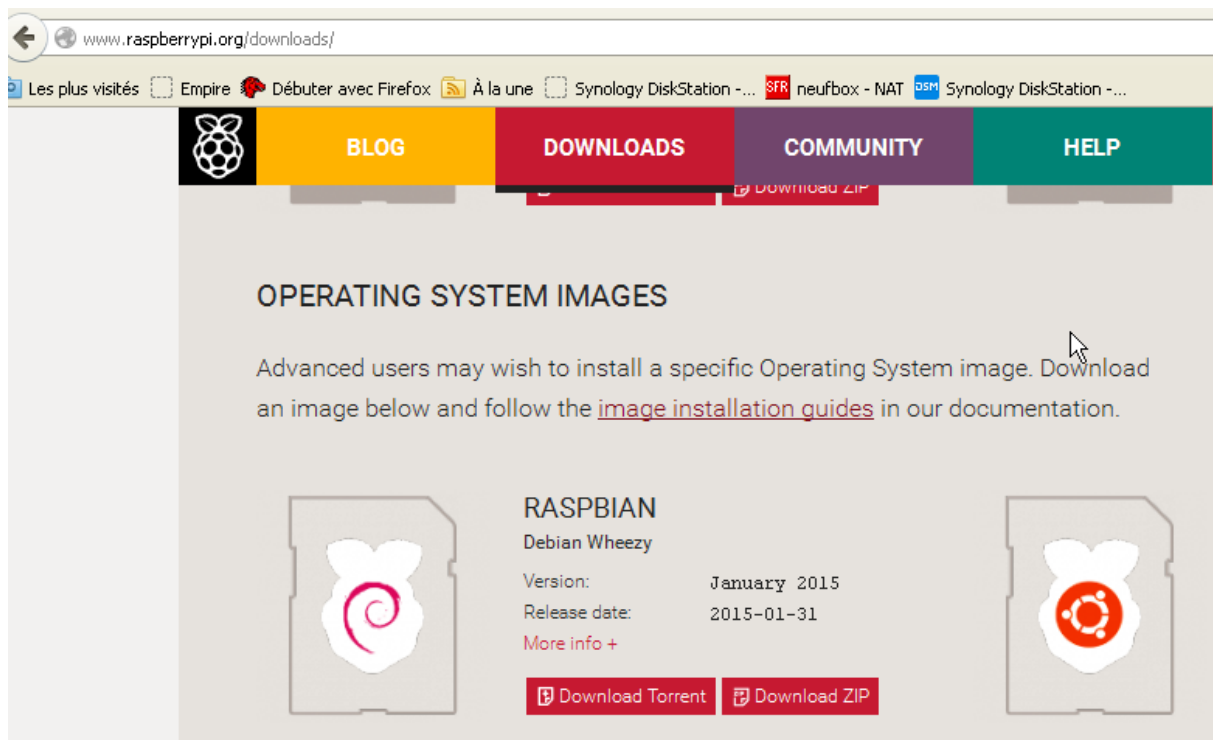
- 1 : données à charger dans Riak CS API par l'application externe
- 2 : Riak CS (Cloud Storage) partitionne les données (1MB en standard)
- 3 : les partitions sont envoyées dans les « Riak Node » (3 au minimum, préconisés)
- 4 : les « Riak Node » répliquent et stockent les données

Riak est un « cloud » sans maître-esclave, chaque nœud est responsable de sa partition et prend le relai si un nœud est indisponible ou un problème de partition réseau. L'éditeur recommande en standard 3 nœuds par réplication.

4. Installation de Raspbian

Nous récupérons l'image de la distribution à l'URL : raspberrypi.org/downloads/

Installation de l'image raspbian :

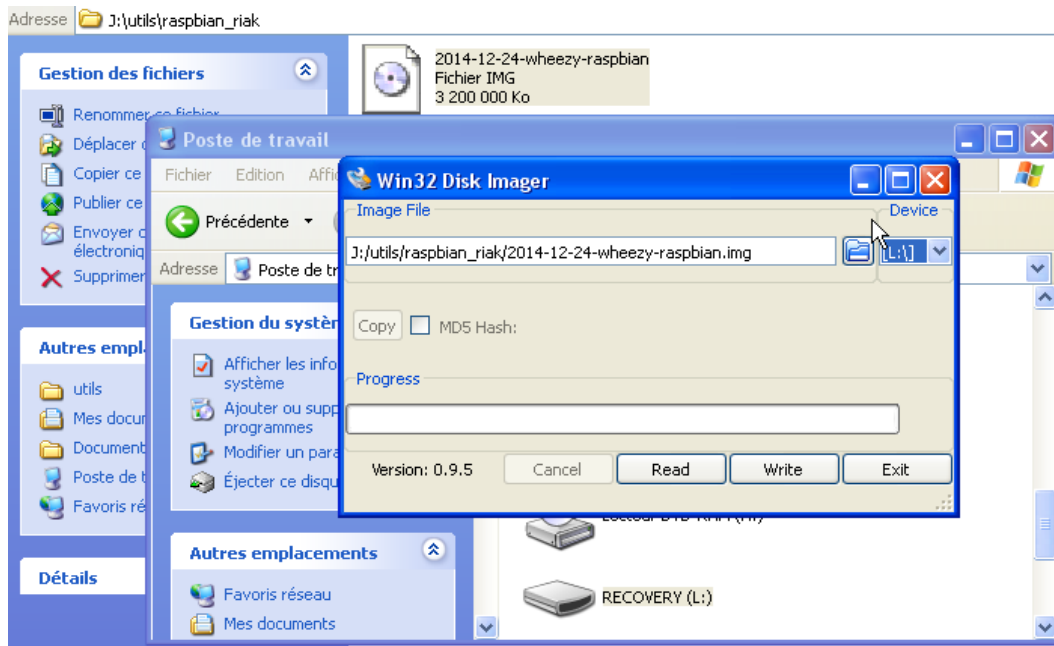


Décompression :

2014-12-24-wheezy-raspbian	983 527 Ko	Archive WinRAR ZIP	31/01/2015 15:08
rehucit	14 825 Ko	Archive WinRAR ZIP	19/01/2015 08:39
PortablePython_3.2.5.1	66 411 Ko	Application	07/01/2015 08:40
BaseCamp_442	53 758 Ko	Application	12/11/2014 10:02
GoogleEarthPluginSetup	860 Ko	Application	12/11/2014 09:36
EPT4-AN14-D32-SD-version0403	209 Ko	Adobe Acrobat Document	12/11/2014 09:17
EPT4-AN14-D33-SD-version0403			14 09:17
EPT4-AN14-D30-SD-version0403			14 09:16
EPT4-AN14-D34-SD-version0403			14 09:16
EPT4-AN14-D35-SD-version0403			14 09:16
EPT4-AN14-D36-SD-version0403			14 09:16
EPT4-AN14-D37-SD-version0403			14 09:16
NU-POST14-D03-SD-version0108			14 10:53
document			14 19:03
install_flashplayer14x32au_mssa_ahh			14 10:05
SpyHunter-Installer			14 10:13
setup(1)			14 21:59
Player			14 20:16
flashget190en			14 20:03
SopCast-3.5.0_by_Wiziwig.tv			14 19:54
Setup			14 20:48
ATT00002			14 11:23
Monkey			14 08:21
lame3.99.5			14 21:52
oggenc2.87-1.3.3-P4			14 21:43
foobar2000_v1.3			14 21:27
calibre-1.16.0			13 09:47
dccedf			13 13:28
Tableau correspondance pneus			13 11:11
URLLink			13 09:40
LAJ-POST13-D11-SD-v01			13 09:34

2014-12-24-wheezy-raspbian	983 527 Ko	Archive WinRAR Z
rehucit	14 825 Ko	Archive WinRAR Z
PortablePython_3.2.5.1	66 411 Ko	Application
BaseCamp_442	53 758 Ko	Application
GoogleEarthPluginSetup	860 Ko	Application
EPT4-AN14-D32-SD-version0403	209 Ko	Adobe Acrobat Dc
EPT4-AN14-D33-SD-version0403	196 Ko	Adobe Acrobat Dc
EPT4-AN14-D30-SD-version0403	220 Ko	Adobe Acrobat Dc
EPT4-AN14-D34-SD-version0403	190 Ko	Adobe Acrobat Dc
EPT4-AN14-D35-SD-version0403	169 Ko	Adobe Acrobat Dc
EPT4-AN14-D36-SD-version0403	183 Ko	Adobe Acrobat Dc
EPT4-AN14-D37-SD-version0403		
NU-POST14-D03-SD-version0108		
document		
install_flashplayer14x32au_mssa_ahh		
SpyHunter-Installer		
setup(1)		
Player		
flashget190en		
SopCast-3.5.0_by_Wiziwig.tv		
Setup		
ATT00002		
Monkey		
lame3.99.5		
oggenc2.87-1.3.3-P4	1 769 Ko	Archive WinRAR Z

Graver l'image par par l'utilitaire Win32DiskImage avec succès.



Démarrage de la distribution avec quelques réglages pour la localisation, réglages du clavier, l'internationalisation, installation du serveur SSH, hostname etc.

```
pi@raspi01: ~
login as: pi
pi@192.168.1.139's password:
Linux raspbri01 3.12.35+ #730 PREEMPT Fri Dec 19 18:31:24 GMT 2014 armv61

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Feb 10 20:31:40 2015 from atos
pi@raspi01 ~ $
```

5. Installation de Riak

L'installation de Riak nécessite l'installation du langage Erlang, elle se fait à partir du dépôt :

deb <http://binaries.erlang-solutions.com/debian/wheezy/contrib>

que nous ajoutons dans la liste du fichier `/etc/apt/sources.list`.

Ajout de la clé publique pour la connexion https :

`wget -O - http://binaries.erlang-solutions.com/debian/erlang_solutions.asc | sudo apt-key add -`

Puis nous pouvons obtenir le paquet du langage Erlang :

`sudo apt-get update`

`sudo apt-get install esl-erlang`

```
pi@raspi01 ~ $ sudo vi /etc/apt/sources.list
pi@raspi01 ~ $ wget -O - http://binaries.erlang-solutions.com/debian/erlang_solutions.asc | sudo apt-key add -
--2015-02-10 22:10:16-- http://binaries.erlang-solutions.com/debian/erlang_solutions.asc
Résolution de binaries.erlang-solutions.com (binaries.erlang-solutions.com)... 31.172.186.53
Connexion vers binaries.erlang-solutions.com (binaries.erlang-solutions.com) [31.172.186.53]:80...connecté.
requête HTTP transmise, en attente de la réponse...302 Found
Emplacement: http://packages.erlang-solutions.com/debian/erlang_solutions.asc [suivant]
--2015-02-10 22:10:16-- http://packages.erlang-solutions.com/debian/erlang_solutions.asc
Résolution de packages.erlang-solutions.com (packages.erlang-solutions.com)... 31.172.186.53
Réutilisation de la connexion existante vers binaries.erlang-solutions.com:80.
requête HTTP transmise, en attente de la réponse...200 OK
Longueur: 3133 (3,1K) [text/plain]
Sauvegarde en : «STDOUT»
100%[=====>] 3 133 --.-K/s ds 0s
2015-02-10 22:10:17 (9,22 MB/s) - envoi vers sortie standard [3133/3133]
OK
```

L'installation de Riak, peut débuter par l'obtention et l'extraction des fichiers sources compressés :

`curl -O http://downloads.basho.com/s3-website-us-east-1.amazonaws.com/riak/1.2/1.2.1/riak-1.2.1.tar.gz`

`tar xzvf riak-1.2.1.tar.gz`

Puis l'utilisation de l'utilitaire `rebar` pour la compilation du paquet :

`cd riak-1.2.1`

`make rel`

```
Compiled src/rekon_resource.erl
Compiled src/riak_control_app.erl
Compiled src/admin_node_stop.erl
Compiled src/admin_node_leave.erl
Compiled src/riak_control_formatting.erl
Compiled src/admin.erl
Compiled src/riak_control_session.erl
Compiled src/admin_gui.erl
Compiled src/admin_cluster_join.erl
Compiled src/admin_routes.erl
Compiled src/admin_services_resource.erl
Compiled src/admin_node.erl
Compiled templates/index.dtl
==> rel (compile)
==> riak-1.2.1 (compile)
Compiled src/etop_txt.erl
==> rel (generate)
```

La source étant compilée, nous pouvons lancer riak :

```
./rel/riak/bin/riak start
```

Utilisation de l'utilitaire Ping pour vérifier le fonctionnement :

```
curl http://localhost:8098/ping
```

Test d'insertion d'une donnée :

```
$ curl -XPUT http://localhost:8098/riak/hello/fr -d 'Allo'
```

```
$ curl http://localhost:8098/riak/hello/fr
```

Allo

```
pi@raspi01 ~/riak-1.2.1 $ ./rel/riak/bin/riak start
pi@raspi01 ~/riak-1.2.1 $ curl http://localhost:8098/ping
OKpi@raspi01 ~/riak-1.2.1 $ curl -XPUT http://localhost:8098/riak/hello/fr -d .Allo.
pi@raspi01 ~/riak-1.2.1 $ curl http://localhost:8098/riak/hello/fr
.Allo.pi@raspi01 ~/riak-1.2.1 $
```

Plus plusieurs réglages IP et hostname etc à remplir ...

Nous répéterons les mêmes opérations pour les deux autres nœuds Riak.

```
sudo dd if="/dev/mmcblk0p1" of="Pi_1.bin"
```

```
sudo dd if="/dev/mmcblk0p2" of="Pi_2.bin"
```

6. Construire les noeuds Riak

Démarrage puis vérification des nœuds Riak :

```

pi@raspi01: ~
login as: pi
pi@192.168.1.139's password:
Linux-raspi01 3.12.35+ #730 PREEMPT Fri Dec 19 18:31:24 GMT 2014 armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 13 22:32:16 2015 from 192.168.1.24
pi@raspi01 ~$ h
pi@raspi01 ~$ h
pi@raspi01 ~$ p
PING rasp02 (192.168.1.139): 56(84) bytes of data:
64 bytes from rasp02: icmp_seq=1 ttl=64 time=0.903 ms
64 bytes from rasp02: icmp_seq=2 ttl=64 time=0.891 ms
64 bytes from rasp02: icmp_seq=3 ttl=64 time=0.890 ms
64 bytes from rasp02: icmp_seq=4 ttl=64 time=0.904 ms
--- rasp02 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.890/0.897/0.904/0.006 ms
pi@raspi01 ~$
pi@raspi02 ~$
pi@raspi02 ~$ sudo shutdown -h now
Broadcast message from root@raspi02 (pts/0) (Fri Feb 13 23:57:40 2015):
The system is going down for system halt NOW!
pi@raspi02 ~$
pi@raspi02 ~$ ping rasp01
PING rasp01 (192.168.1.139) 56(84) bytes of data:
64 bytes from rasp01: icmp_seq=1 ttl=64 time=0.961 ms
64 bytes from rasp01: icmp_seq=2 ttl=64 time=0.955 ms
64 bytes from rasp01: icmp_seq=3 ttl=64 time=0.952 ms
--- rasp01 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 0.948/1.203/1.709/0.357 ms
pi@raspi02 ~$
pi@raspi03 ~$
pi@raspi03 ~$ sudo shutdown -h now
Broadcast message from root@raspi03 (pts/0) (Fri Feb 13 23:57:40 2015):
The system is going down for system halt NOW!
pi@raspi03 ~$
pi@raspi03 ~$ ping rasp01
PING rasp01 (192.168.1.139) 56(84) bytes of data:
64 bytes from rasp01: icmp_seq=1 ttl=64 time=0.903 ms
64 bytes from rasp01: icmp_seq=2 ttl=64 time=0.891 ms
64 bytes from rasp01: icmp_seq=3 ttl=64 time=0.890 ms
64 bytes from rasp01: icmp_seq=4 ttl=64 time=0.904 ms
--- rasp01 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.890/0.897/0.904/0.006 ms
pi@raspi03 ~$ ping rasp02
PING rasp02 (192.168.1.139) 56(84) bytes of data:
64 bytes from rasp02: icmp_seq=1 ttl=64 time=0.961 ms
64 bytes from rasp02: icmp_seq=2 ttl=64 time=0.955 ms
64 bytes from rasp02: icmp_seq=3 ttl=64 time=0.952 ms
--- rasp02 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 0.948/1.203/1.709/0.357 ms
pi@raspi03 ~$

```

Configuration de l'adresse IP de chaque nœud Riak dans le fichier de la machine virtuelle Erlang :

\$RIAK_HOME/rel/riak/etc/vm.args :

```

pi@raspi01: ~/riak-1.2.1/rel/riak
## Name of the riak node
#-name riak@127.0.0.1
-name riak@192.168.1.139

pi@raspi02: ~/riak-1.2.1/rel/riak
## Name of the riak node
#-name riak@127.0.0.1
-name riak@192.168.1.53

## Cookie for distributed erlang. All nodes in the same cluster

pi@raspi03: ~/riak-1.2.1/rel/riak
## Name of the riak node
#-name riak@127.0.0.1
-name riak@192.168.1.187

## Cookie for distributed erlang. All nodes in the same cluster
## should use the same cookie or they will not be able to communicate.

```

Projet VLS NFE 204

Configuration de l'adresse IP dans le fichier de configuration générale:

```
$RIAK_HOME/rel/riak/etc/etc/app.config
```

Redémarrage de chaque nœud :

```
pi@raspi01: ~/riak-1.2.1/rel/riak
--setcookie riak
## Heartbeat management; auto
## (Disabled by default..use
##-heart

## Enable kernel poll and a
+K true
+K 64

## Treat error_logger warning
+W w

## Increase number of concurr
-env ERL_MAX_PORTS 4096

## Tweak GC to run more often
-env ERL_FULLSWEEP_AFTER 0

## Set the location of crash
-env ERL_CRASH_DUMP ./log/er

## Raise the ETS table limit
-env ERL_MAX_ETS_TABLES 8192

## Begin SSL distribution ite
+K

## To enable SSL encryption o
## un-comment the three lines
## point to correct PEM data

## -proto dist inet ssl
## -ssl dist_opt client cert
## -ssl dist_opt server cert

## End SSL distribution item

pi@raspi01: ~/riak-1.2.1/rel/riak
eth0
inet addr:192.168.1.139
UP BROADCAST RUNNING
RX packets:1344 err
TX packets:1021 err
collisions:0 lg fi
RX bytes:95242 (93

pi@raspi02: ~/riak-1.2.1/rel/riak
login as: pi
pi@192.168.1.53's password:
Linux raspberrypi 3.12.35+ #730 PREEMPT Fri Dec 19 18:31:24 GMT 2014 armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/*-copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 13 21:36:09 2015 from 192.168.1.24

pi@raspi02: ~$ ping raspberrypi
PING raspberrypi (192.168.1.139): 56(84) bytes of data:
64 bytes from raspberrypi (192.168.1.139): icmp_req=1 ttl=64 time=0.908 ms
64 bytes from raspberrypi (192.168.1.139): icmp_req=2 ttl=64 time=0.949 ms
64 bytes from raspberrypi (192.168.1.139): icmp_req=3 ttl=64 time=0.925 ms
^C
--- raspberrypi ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.500/0.580/0.640/0.040 ms

pi@raspi03: ~/riak-1.2.1/rel/riak
drwx----- 3 pi pi 4096 sept. 26 2012 doc
drwx----- 2 pi pi 4096 f1@vrr. 13 23:15 ebin
-rw-r--r-- 1 pi pi 10175 sept. 26 2012 LICENSE
-rw-r--r-- 1 pi pi 6835 sept. 26 2012 Makefile
-rw-r--r-- 1 pi pi 276 sept. 26 2012 NOTICE
-rw-r--r-- 1 pi pi 1254 sept. 26 2012 orgbatch.el
drwx----- 8 pi pi 4096 sept. 26 2012 package
-rw-r--r-- 1 pi pi 11537 sept. 26 2012 README.org
-rw-r--r-- 1 pi pi 109842 sept. 26 2012 rebar
-rw-r--r-- 1 pi pi 592 sept. 26 2012 rebar.config
drwx----- 5 pi pi 4096 f1@vrr. 13 23:16 rel
drwx----- 2 pi pi 4096 sept. 26 2012 releasenotes
-rw-r--r-- 1 pi pi 14853 sept. 26 2012 RELEASE-NOTES.md
drwx----- 2 pi pi 4096 sept. 26 2012 src
-rw-r--r-- 1 pi pi 3475 sept. 26 2012 THANKS

pi@raspi03: ~/riak-1.2.1 $ ifconfig eth0
eth0
Link encap:Ethernet HWaddr b8:27:eb:d3:64:f5
inet addr:192.168.1.187 Bcast:192.168.1.255 Masque:255.255.255.0
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:490 errors:0 dropped:0 overruns:0 frame:0
TX packets:371 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 lg file transmission:1000
RX bytes:36128 (35.2 KiB) TX bytes:40991 (40.0 KiB)

pi@raspi01: ~/riak-1.2.1/rel/riak $ vi etc/vm.args
pi@raspi01: ~/riak-1.2.1/rel/riak $ vi etc/vm.args
pi@raspi01: ~/riak-1.2.1/rel/riak $ vi etc/app.config
pi@raspi01: ~/riak-1.2.1/rel/riak $ vi etc/app.config
pi@raspi01: ~/riak-1.2.1/rel/riak $ bin/riak start
pi@raspi01: ~/riak-1.2.1/rel/riak $

pi@raspi03: ~/riak-1.2.1/rel/riak $ vi etc/vm.args
pi@raspi03: ~/riak-1.2.1/rel/riak $ vi etc/vm.args
pi@raspi03: ~/riak-1.2.1/rel/riak $ vi etc/app.config
pi@raspi03: ~/riak-1.2.1/rel/riak $ bin/riak start
pi@raspi03: ~/riak-1.2.1/rel/riak $
```

Ajout d'un nœud :

```
./bin/riak-admin cluster join
```

Puis vérification des membres :

```
./bin/riak-admin member-stat
```

Puis validation du plan :

```
./bin/riak-admin cluster plan
```

```
./bin/riak-admin cluster commit
```

```
yiv@yiv-laptop:~/TIA@C@chargements/riak-1.2.1/rel/riak$ bin/riak-admin member-status
===== Membership =====
Status      Ring    Pending  Node
-----
valid       25.0%   --       'riak@192.168.1.139'
valid       25.0%   --       'riak@192.168.1.187'
valid       25.0%   --       'riak@192.168.1.40'
valid       25.0%   --       'riak@192.168.1.53'
-----
Valid:4 / Leaving:0 / Exiting:0 / Joining:0 / Down:0
```

7. Insertion des données dans les nœuds Riak

Dans la terminologie Riak, l'entité « bucket » peut être comparée à la table dans le monde relationnel ou encore à au dossier dans l'arborescence d'un système de fichiers.

Nous nous contenterons d'utiliser les valeurs de configuration par défaut.

L'acquisition des données étant par station, la forme des données JSON à insérer est de la forme :

```
{
  { <num_station>_horodatage } :
  { « données dynamiques envoyées par l'API RESTful JC DECAUX » }
}
<num_station>_horodatage est la clé
{ « données dynamiques envoyées par l'API RESTful JC DECAUX » } la valeur
```

Chaque « bucket » se voit attribuer le numéro de station.

La clé attribuée est le résultat des concaténations :

<num_station>_<date>_<heure>_<minute>

Ci-dessous l'exemple d'insertion des données pour la commande curl :

```
curl -v -XPUT -d '
{
  '4001_2015-02-14_13' :
  {'lat': 48.85398414441304, 'lng': 2.349380045291025}, 'status': 'OPEN', 'contract_name': 'Paris',
  'last_update': 1423917725000, 'bonus': False, 'number': 4001, 'bike_stands': 32}
}' -H "Content-Type: application/json"
http://192.168.1.40:8098/buckets/4001/keys/4001_2015-02-14_13?returnbody=true
```

Pour automatiser l'insertion des données, nous utilisons un script python avec l'api Riak (voir l'annexe pour le détail du script).

8. Quelques statistiques

Cette partie est très courte car nous ne disposons pas pour l'instant assez de données (sur une semaine complète pour les relevés de chaque heure de 07H00 à 22H00 ce qui nous empêche de faire l'algorithme de prédiction annoncé précédemment.

```
yiv@yiv-laptop:~/NFE204/projet/scripts$ curl -X GET
```

```
http://192.168.1.139:8098/buckets/5001/keys/5001_2015-02-22_09
```

```
"{'name': '05001 - SAINT GERMAIN HARPE', 'contract_name': 'Paris', 'bike_stands': 43, 'address': '32 RUE DE LA HARPE - 75005 PARIS', 'position': {'lng': 2.3437045367227731, 'lat': 48.851581340028176}, 'number': 5001, 'bonus': False, 'status': 'OPEN', 'banking': True, 'available_bikes': 36, 'last_update': 1424592230000, 'available_bike_stands': 7}"
```

```
yiv@yiv-laptop:~/NFE204/projet/scripts$ curl -X GET
```

```
http://192.168.1.139:8098/buckets/5001/keys/5001_2015-02-23_09
```

```
"{'status': 'OPEN', 'available_bikes': 43, 'contract_name': 'Paris', 'name': '05001 - SAINT GERMAIN HARPE', 'address': '32 RUE DE LA HARPE - 75005 PARIS', 'available_bike_stands': 0, 'number': 5001, 'bonus': False, 'last_update': 1424678696000, 'bike_stands': 43, 'position': {'lat': 48.851581340028176, 'lng': 2.3437045367227731}, 'banking': True}"
```

```
yiv@yiv-laptop:~/NFE204/projet/scripts$ curl -X GET
```

```
http://192.168.1.139:8098/buckets/5001/keys/5001_2015-02-24_09
```

```
"{'available_bikes': 20, 'address': '32 RUE DE LA HARPE - 75005 PARIS', 'bonus': False, 'number': 5001, 'name': '05001 - SAINT GERMAIN HARPE', 'banking': True, 'position': {'lng': 2.3437045367227731, 'lat': 48.851581340028176}, 'status': 'OPEN', 'last_update': 1424764676000, 'bike_stands': 43, 'contract_name': 'Paris', 'available_bike_stands': 21}"
```

```
yiv@yiv-laptop:~/NFE204/projet/scripts$ curl -X GET
```

```
http://192.168.1.139:8098/buckets/5001/keys/5001_2015-02-26_09
```

```
"{'number': 5001, 'available_bikes': 18, 'banking': True, 'last_update': 1424937572000, 'available_bike_stands': 25, 'contract_name': 'Paris', 'address': '32 RUE DE LA HARPE - 75005 PARIS', 'bonus': False, 'status': 'OPEN', 'name': '05001 - SAINT GERMAIN HARPE', 'bike_stands': 43, 'position': {'lng': 2.3437045367227731, 'lat': 48.851581340028176}}"
```

```
yiv@yiv-laptop:~/NFE204/projet/scripts$ curl -X GET
```

```
http://192.168.1.139:8098/buckets/5001/keys/5001_2015-02-26_09
```

```
"{'number': 5001, 'available_bikes': 18, 'banking': True, 'last_update': 1424937572000, 'available_bike_stands': 25, 'contract_name': 'Paris', 'address': '32 RUE DE LA HARPE - 75005 PARIS', 'bonus': False, 'status': 'OPEN', 'name': '05001 - SAINT GERMAIN HARPE', 'bike_stands': 43, 'position': {'lng': 2.3437045367227731, 'lat': 48.851581340028176}}"yiv@yiv-
```

```
laptop:~/NFE204/projet/scripts$ curl -X GET
```

```
http://192.168.1.139:8098/buckets/5001/keys/5001_2015-02-27_09
```

```
"{'banking': True, 'contract_name': 'Paris', 'name': '05001 - SAINT GERMAIN HARPE', 'position': {'lat': 48.851581340028176, 'lng': 2.3437045367227731}, 'address': '32 RUE DE LA HARPE - 75005 PARIS', 'bonus': False, 'available_bike_stands': 26, 'status': 'OPEN', 'available_bikes': 16, 'bike_stands': 43, 'last_update': 1425024172000, 'number': 5001}} »
```

9. Conclusion

Nous pensons qu'à partir des données brutes collectées, la fiabilité des prévisions seraient « moyenne » sans tenir compte d'autres paramètres tels que la météo, les vacances etc.

Citons le logiciel Qucit qui exploite les données de quatre années de fonctionnement de ce réseau de vélos en libre-service dans l'agglomération bordelaise, pour établir les prévisions d'encombrement des stations en fonction des heures (source les échos).

10. Scripts shell et python

Le script Shell permet de découper par arrondissement dans Paris et par commune si hors de Paris à partir du fichier des données statiques fourni par JC Decaux. A partir de ces fichiers découpés, le script Python se charge de prendre en paramètre et alimenter les données dynamiques du service RESTful exposé à l'URL :

<https://api.jcdecaux.com/vls/v1/stations/nnnnn?contract=Paris&apiKey=XXXX>

nnnnn étant le numéro de station et XXXX la clé d'API fourni par JC Decaux

Détails du script de découpage :

```
#!/usr/bin/ksh

#set -x

REP=`pwd`
REP_ST=$REP/lst_st
echo "nb param : $#"
```

echo "param 1 \$1"

```
if [ "$1" = "" ]
then
    echo "Pas de fichier parametre en entree !"
    exit 1
else
    NOM_FIC="$1"
    echo "nom fichier : $NOM_FIC"
    cat ${NOM_FIC}|sed "s/^(^([0-9]*)),.*1/g" > $REP/lst_stations_paris.txt
```

```
# trie le fichier des stations avec l'option numerique
sort -n $REP/lst_stations_paris.txt > $REP/lst_stations_paris_trie.txt
# Decoupage des stations suivant des arrondissements dans Paris
# et par commune autour de Paris
# Attention l'entree du fichier est $REP/lst_stations_paris_trie.txt
# en fin de boucle while
typeset -i lig
while read lig
do
    if [ -f "${REP_ST}/lst_st_${lig/1000}.txt" ]
    then
        touch ${REP_ST}/lst_st_${lig/1000}.txt
    fi
    echo $lig >> ${REP_ST}/lst_st_${lig/1000}.txt
done < $REP/lst_stations_paris_trie.txt
fi
```

Résumé du script Python :

```
# Script d'acquisition des données Velib JC Decaux
# -*- coding:utf-8 -*-
import urllib.request, urllib.error, urllib.parse
import json
import os
import sys
import datetime
import time
import argparse

# Cette partir permet au programme Python de prendre le fichier en entrée
parser = argparse.ArgumentParser()
parser.add_argument('infile', nargs='?', type=argparse.FileType('r'),...
parser.add_argument("-f", "--file", action="count", default=0,...
```

```
args = parser.parse_args()
answer = "Sortie de la liste des stations velib"

# Initialisation et insertion de la liste à partir du fichier d'entrée
lst_stations = []
lst_st = open(args.infile.name, "rU", encoding='cp1252').readlines()
for line in lst_st:
    ...

# Contrôle des paramètres d'entrée
if args.file > 1:
    ...
else:
    print("Nb d'argv passé à l'interpréteur Python : ",sys.argv)

# obtention de la clef d' API dans ~/.velib
with open(os.path.expanduser('~/.velib'), 'r') as f:
    key = f.read()

def collecting_data (station      = 4020,
                    delayms      = 1000,
                    outfile       = "velib_data.txt",
                    single_file   = True,
                    stop_datetime = None,
                    log_every     = 10) :
    """
    collects data for a period of time

    :param contract: contract name, see :term:`_contracts`
    :param delayms: delay between two collections (in ms)
    :param outfile: write data in this file (json)
    :param single_file: True: one file, else, many files with timestamp as a suffix
```

```
:param    stop_datetime:  if None, never stops, else stops when the date is reached
:param    log_every:      print something every <log_every> times data were collected
:return:          list of created file
```

```
.....
```

```
....
```

```
def geturl(path):
```

```
.....
```

```
    Obtention de l'URL complet à partir de l'URL court du service REST
```

```
:param    path:          url court
```

```
.....
```

```
    delim = '&' if '?' in path else '?'
```

```
    return "https://api.jcdecaux.com/vls/v1/{0:s}{1:s}apiKey={2:s}".format(path, delim, key)
```

```
def get(path):
```

```
.....
```

```
    Retourne les données JSON à partir de l'URL long
```

```
:param    path:          url long
```

```
.....
```

```
    url = geturl(path)
```

```
    print(urllib.request.urlopen(url).read().decode('utf-8'))
```

```
    return json.loads(urllib.request.urlopen(url).read().decode('utf-8'))
```

```
# Filtrage seulement pour le contrat Paris
```

```
filter(lambda d: d['name'] == 'Paris', get('contracts'))
```

```
...
```

```
# Partie principale
```

```
for num_st in lst_stations :
    print("%s" % num_st)
    nom_fich = "velib_data" + "_" + str(num_st)
# chemin à modifier (mettre en paramètre)
    chemin = "/home/yiv/NFE204/projet/"
    nom_fich_ext = chemin + nom_fich + ".txt"
    print("%s %s" % ("nom_fich_ext : ", nom_fich_ext))
    for i in range(st_1,st_n) :
        if i % num_st == 0 :
            print("%s %s" % (str(i), str(num_st)))
            #fin_time_collect = datetime.datetime.now() +
datetime.datetime.fromtimestamp(10000.0)
            interval_time = datetime.datetime.fromtimestamp(0.)
            print(interval_time)
            fin_time_collect = datetime.datetime.now() + datetime.timedelta(seconds=9)
            print(fin_time_collect)
            collecting_data(num_st, 10000, outfile=nom_fich_ext,
stop_datetime=fin_time_collect)
```

Le code complet se trouve dans le fichier ci-joint



my_station_data_col
_velib_py3_param_3.